



Dissertation

A Formal Verification Framework for CRDTs in Athena

Leonardo Ángel

November 19, 2025

This thesis is submitted in partial fulfillment of the requirements for a degree of Master in Systems and Computing Engineering (MISIS).

Thesis Committee:

Prof. Nicolás Cardozo (Promotor) Universidad de los Andes, Colombia

A Formal Verification Framework for CRDTs in Athena

© 2025 Leonardo Ángel

Systems and Computing Engineering Department

FLAG lab

Faculty of Engineering

Universidad de los Andes

Bogotá, Colombia

This manuscript has been set with the help of TEXSHOP and PDFLATEX (with BIBTEX support). Bugs have been tracked down in the text and squashed thanks to *Bugs in Writing* by Dupré [2] and *Elements of Style* by Strunk and White [16].

DEDICATION

Abstract

Ensuring consistency in distributed systems, particularly in Conflict-Free Replicated Data Types (CRDTs), poses significant challenges, especially in environments prone to Byzantine faults. Collaborative editing systems such as peer-to-peer (P2P) platforms exemplify these challenges, where nodes are vulnerable to malicious behavior and data inconsistencies due to transient disconnections. Designing such data types is non-trivial as subtle choices in the semantics of operations can lead to unintended divergence. Formal verification provides a principled way to address these complexities by proving that a CRDT satisfies properties like convergence and commutativity. Existing work shows how individual CRDTs can be verified using theorem provers.

Our proposal offers a structured approach for CRDT verification by developing a modular framework in the Athena proof assistant. The framework facilitates the specification of both the syntax and operational semantics of CRDTs and guides the proof of key properties such as convergence, commutativity, and idempotence. By identifying and encapsulating common patterns in CRDT design and verification. It is suited to developers who are exploring or prototyping new CRDT variants and need a precise and checkable way to reason about their correctness. While the framework does not cover all possible CRDT designs. It takes a modest step toward making formal verification more approachable in this domain.

1 Introduction

Distributed systems replicate data across nodes to allow performance and availability. However, keeping all replicas consistent when dealing with concurrency, delays, and network partitions is a significant challenge. Traditional solutions that enforce strong consistency usually rely on consensus protocols and global coordination, which can be expensive and unstable under failure scenarios. As an alternative solution, *eventual consistency* is proposed. In this case updates may be applied independently at different replicas and reunified later in the process.

Conflict-Free Replicated Data Types (CRDTs) were defined to formalize this idea. CRDTs are data structures that allow updating concurrently and can be applied without coordination but still guarantee that all the replicas eventually reach the same state, as long as they receive the same set of updates. This property, known as *Strong Eventual Consistency* (SEC) is achieved through the design of asynchronous update and merge operations which have to satisfy algebraic conditions including commutativity, associativity, idempotence, and monotonicity [15].

Designing such data types is not always straightforward. Even small design choices, for instance how to handle removals in sets, can introduce issues that may conduct to divergence or break the intended semantics. Formal verification provides a solution to avoid behavioral errors by proving that a CRDTs satisfy properties like convergence and commutativity.

Prior work has demonstrated how to specify and verify CRDTs in theorem provers like Isabelle/HOL [20], establishing rigorous correctness guarantees for individual CRDT implementations. This thesis explores a structured approach by suggesting a more general purpose framework in the Athena proof assistant for specifying and verifying multiples types of CRDTs. The goal is to reduce the effort of building dedicated frameworks to verify each independent implementation of CRDTs by identifying common patterns in their design and behavior, using these patterns in reusable components. Our proposed framework aims to support the definition of state-based

CRDTs and helps express their semantics in a modular way. It also provides the structure to reason about properties like convergence and idempotence.

The contribution is not a complete or fully automated system but a set of structures that make it easier to work with CRDTs in Athena. In particular, it is aimed at situations where developers want to experiment with CRDT variants and need a way to express their behaviors precisely and check key correctness properties. While this framework is limited in scope and not intended to cover every possible CRDT design; it offers a modest step toward making formal verification more practical in this domain.

Needs a paragraph discussing the validation and results. Needs a paragraph discussing the thesis outline (structure and what you find in each chapter).

2 Related work

Have to do: Add glue text here explaining what this chapter does.

2.1 Conflict-Free Replicated Data Types and eventual consistency

To provide context for CRDT verification, we start by reviewing the main ideas related to Conflict-Free Replicated Data Types (CRDTs) and their role in Eventually Consistent systems. Replication is a typical strategy for achieving high availability and fault tolerance in distributed systems, however maintaining the consistent aspect of replicated data can be expensive. Traditional approaches that require strong consistency rely on global consensus which may create performance conflicts with availability under network partitions [15]. The CAP theorem states that a distributed system can not guarantee availability, partition tolerance and strong consistency at the same time, thus systems often weaken consistency in favour of availability and partition tolerance [20].

Shapiro et al. formalised this trade-off by introducing the model of *Strong Eventual Consistency* (SEC). In the context of a eventually consistent system, updates when executed at one replica are propagated to other replicas asynchronously, possibly in different orders: once all updates have been distributed, all replicas must converge to the same state [15]. Under SEC two properties are required: (1) *eventual delivery* which means every update that is delivered to one specific replica eventually is delivered to all replicas and (2) *convergence*, meaning that replicas that have delivered the same updates eventually reach equivalent states [12]. CRDTs are a class of replicated data structures aimed to satisfy SEC by construction. They mix conflict resolu-

tion into the data type itself allowing updates in a concurrent state to be applied locally and merged deterministically without any coordination. Early work observed that both state-based and operation-based CRDTs can be specified by algebraic conditions: in *state-based* CRDTs, the state space is a join-semilattice and the merge operation computes the least upper bound requiring commutativity, associativity and idempotence [20, 15], in *operation-based* CRDTs, updates (operations) are broadcast in causal order and they have to commute so that replicas that apply the same set of operations reach the same state [12].

Designing CRDTs that guarantee SEC is non trivial. One of the most simple examples such as grow-only counter can be specified using per replica counters and merged by taking component-wise maxima [20]. Some other and more complex data types such as sets that support both addition and removal require semantics designed to avoid anomalies like resurrecting deleted elements. This innovative work inspired research on CRDT design and verification. **This section is missing a clear definition of what is operation-based CRDTs and state-based CRDTs, which will be the focus of this work. But need to add them both and make it clear what the difference is**

2.2 Formal specification and verification frameworks

Early verification efforts were conducted to provide machine-checked proofs that CRDT designs actually satisfy SEC. Zeller, Bieniusa and Poetzsch-Heffter developed an Isabelle/HOL framework for specifying and verifying CRDTs [20]. Their framework models an specific number of replicas. Their model abstracts over payloads, update operations and merge functions, representing each replica's state and history with associated metadata and version vectors. This framework enabled the first machine checked convergence proofs for some CRDTs.

While the Isabelle framework is practical, it requires significant manual effort. To reduce this detail, Nagar and Jagannathan formulated a parameterised verification apporach that obtains convergence conditions from a given consistency model [11]. Their engine shows that not all operations must commute to guarantee convergence and produces automated proofs for CRDTs under various weak consistency policies [11].

Separation logic has been transformed to verify CRDT implementations. Nieto et al. intro-

duced OpLib and RcbLib libraries that allow modular verification for operation-based CRDTs using Aneris distributed separation logic. These libraries abstract away low-level concurrency concerns and provide mechanised Coq proofs for a collection of replicated data structures [12]. The approach was later extended to state-based CRDTs because states grow monotonically to include all updates, unified specifications can be written without distinguishing between state and operation based designs [13].

Recent research also uses type systems, domain-specific languages and program synthesis to guarantee CRDT correctness by construction. Propel is a Scala-embedded language whose type system reasons about algebraic properties such as commutativity, associativity and idempotence, ensuring that merge functions satisfy these laws [19]. VeriFx takes a complementary approach by integrating verification into a high-level language: programmers implement replicated data types and specify correctness properties, and the language automatically proves those properties and converts verified implementations to mainstream languages [14]. Liang and Feng observed that strong eventual consistency does not ensure functional correctness and proposed Abstract Converging Consistency (ACC), which augments CRDT specifications with abstract atomic operations, enabling modular verification of both implementations and clients [8]. Katara automatically generates verified CRDTs from sequential data types by encoding correctness conditions into SMT logic and searching for suitable semilattice representations [6].

2.3 Byzantine fault tolerance and CRDTs

Kleppmann showed that augmenting CRDT algorithms with signatures, hashes and vector clocks can tolerate an arbitrary number of Byzantine replicas while preserving strong eventual consistency [5]. More recent work identifies CRDT classes that prevent equivocation; for example, Jacob, Bayreuther and Hartenstein observed that the Matrix Event Graph stores updates in a hash-based directed acyclic graph and tolerates equivocation without additional coordination [4]. Such results highlight both the feasibility and the limits of building Byzantine-tolerant CRDTs.

Tschudin proposed a “connectionless” grow-only set CRDT that operates over lossy, reorderable channels without the necessity of any persistent connections between nodes. This CRDT uses a single message type and incremental updates to spread state, making it suitable for ap-

plications such as peer-to-peer compression dictionaries where maintaining per-node connections is unfeasible [17]. Such innovations widen the scope of the CRDT paradigm to networks where communication can be intermittent. Additional mechanisms may be necessary to handle message loss and ensure eventual convergence.

2.4 Verification in actor and concurrency frameworks

Musser and Varela developed a structured reasoning framework for actor systems using the Athena proof language. They established abstract theorems about fairness, persistence, message delivery and progress that hold for any actor system, independently of implementation details [10]. Their work demonstrates that high-level algebraic reasoning can give general properties such as guaranteed message delivery and monotonicity of actor states [10]. These results motivate the use of high-level proof environments when reasoning about replicated data structures.

Formalisation of concurrent languages has also lead to the development of modular semantics used in many proof frameworks. Bereczky, Horpácsi and Thompson presented a machine-checked formalisation of concurrent Core Erlang using modular frame stack semantics [1]. They proved properties such as determinism, confluence and program equivalence [1] providing a foundation for verifying concurrency properties of actor-based programming languages.

The actor model has also inspired languages for dynamic and open systems. For instance, SALSA compiles to Java and provides primitives for actor creation, asynchronous messaging, state updates, continuations and universal naming to coordinate distributed actors [18]. It emphasises disconected message passing and mobility, repeating with the asynchronous update and merge semantics of CRDTs and illustrate how actor based languages can support reasoning about distributed computations.

2.4.1 Program Synthesis and Automated CRDT Generation

Katara (2022) introduced a novel approach to synthesizing CRDTs using verified lifting, allowing developers to generate CRDT implementations automatically from sequential data types. This synthesis approach leverages lightweight ordering constraints and inductive invariants enabling the automated generation and verification of new CRDTs [6]. This automation addresses

the inherent difficulty in designing CRDTs manually and offers a way to scale CRDTs to new applications.

2.4.2 Comparative Analysis: Defining Grammar, Rule Inference, and Formal Verification

In order to provide a clearer understanding of the landscape of CRDT research, we summarize the state-of-the-art in a 2.1 that highlights the following dimensions for each paper: (1) Definition of CRDT grammar, (2) Inference of rules or operations, (3) Implementation in programming languages, (4) Formal verification of properties, and (5) Byzantine Fault Tolerance (BFT).

tabularx booktabs

Table 2.1: Comparison of CRDT research proposals

Paper/Research	**Grammar**	**Rule**	**Language**	**Verification**	**BFT**
Zeller et al. (2014)	Yes	Yes	Isabelle/HOL	Yes	No
Katara (2022)	Yes	Yes	Automatic (synthesis)	Yes (automated)	No
Syncordian (Sanabria & Cardozo, 2022)	Yes	Yes	Elixir	Yes (Athena)	Yes
Kleppmann (2022)	No	Yes	Not specified	No	Yes
Nagar & Jagannathan (2019)	Yes	Yes	Not specified	Yes (automated)	No
Sanjuan et al. (2020, Merkle-CRDTs)	No	Yes	IPFS	No	No
Liang & Feng (2021)	Yes	Yes	Not specified	Yes (modular)	No

2.4.3 Discussion

From the comparative analysis, it becomes evident that formal verification remains one of the most significant challenges in CRDT development. Early frameworks like Isabelle/HOL provide rigorous correctness guarantees but require expert involvement. Automated approaches such as parameterised verification [11], separation logic libraries [12], type systems [19] and domain-specific languages [14] lower the burden of proof but they are often tailored to specific models. Recent efforts to provide abstractions for functional correctness such as ACC [8], and equivocation

tolerance in Byzantine environments [4] suggest that when CRDTs are used as building blocks in complex systems richer correctness specifications are required.

While many researchers have provided grammars and rules for CRDT operations, fewer have focused on automating the verification process. Tools like Katara and the verification engine developed by Nagar and Jagannathan represent important advances in reducing the complexity of designing and verifying CRDTs [6, 11]. Work on Byzantine fault tolerance shows that many CRDT algorithms can be extended to handle malicious replicas through cryptographic techniques [5] but systematic verification of such extensions remains scarce. While Byzantine Fault Tolerance is gaining importance with the rise of decentralized systems, only a few CRDT implementations focus on this aspect. Kleppmann's work on Byzantine Fault Tolerance provide significant contributions in this regard, but much remains to be explored in ensuring that CRDTs can function reliably in adverse environments [5].

2.4.4 Conclusion

The research on CRDTs has advanced significantly over the last decade, particularly in defining the grammar and rules for these data structures and verifying their convergence and consistency. However, challenges like automating the verification process and extending CRDTs to tolerate Byzantine faults remain. As distributed systems continue to evolve, the ability to design, implement, and verify CRDTs will be crucial for building scalable, resilient systems.

xcolor listings

3 Vector Clocks for State-Based CRDTs

Vector clocks provide an algebra for reasoning about causality in asynchronous systems. We follow the standard path: (*1*) Lamport's *happens-before* relation $\rightarrow$ as a partial order for events [7]; (*2*) vector timestamps ("vector clocks") introduced independently by Fidge and Mattern to represent a causal structure and offer maintenance rules in message-passing systems [3, 9] and (*3*) the semilattice perspective that reinforces the state-based CRDT convergence [15].

3.1 Mathematical model and notation

Notation provenance. We use the standard timestamp notation of Fidge and Mattern [3] where clocks are finitely supported maps $C : \mathsf{Ide} \to \mathbb{N}$ (missing entries are 0), the causal preorder is component-wise $C \preceq D \iff \forall i.\, C(i) \leq D(i)$, and the merge is the pointwise maximum $(C \sqcup D)(i) = \max\{C(i), D(i)\}$. This definition refines Lamport's happens–before relation and makes possible the increment rules [9, 7]. The semilattice view (idempotent, commutative and associative) is the one view used in state–based CRDT convergence proofs [15].

We adopt the usual functional notation for vector timestamps from Fidge and Mattern: a clock is a mapping from identifiers of processes to natural numbers having absent components with value 0 [3, 9]. Causality is read component-wise, refining Lamport's logical clocks [7].

Clocks. Let Ide be a finite or countable set of actor identifiers and $\mathbb{N}$ the naturals. A *vector clock* is a finitely supported map $C : \mathsf{Ide} \to \mathbb{N}$; where $i \notin \mathrm{dom}(C)$ we read $C(i) = 0$ [3, 9].

Order and join. Following the standard pointwise definition, we write

$$C \preceq D \iff \forall i \in \mathsf{Ide}.\, C(i) \leq D(i) \qquad \text{and} \qquad (C \sqcup D)(i) = \max\{C(i), D(i)\}.$$

Then $(\mathcal{C}, \preceq, \sqcup)$ is a join-semilattice: $\sqcup$ is commutative, associative, idempotent, and is the least upper bound for $\preceq$. The strict causal test is:

$$C \prec D \iff C \preceq D \ \wedge \ \exists i.\, C(i) < D(i),$$

and concurrency is obtained by incomparability: $C \parallel D$ iff neither $C \preceq D$ nor $D \preceq C$ holds [7, 3, 9].

Operational rules A process increments its own component on local events, carry its current vector on send and joins using component-wise maximum on receive [3, 9]. For state-based CRDTs this implies that metadata evolves by inflation ($C \preceq C \sqcup D$), merges are idempotent, commutative and associative and repeated deliveries do not change the eventual outcome.

Have to do: Additionally add a footnote saying that the implementation is available in the repo, and add a link to the repo Have to do: This section on the code needs to explain a bit more what the functions are, and there use for the verification framework

3.2 Our approach in `Athena`

3.1 show the definition of vector clocks, developed in `Athena`:

Have to do: add text explaining what parts of the code corespond to which parts of the definition

```
1
2  module VCCore {
3
4  define ++ := FMap.++           # alias for map extension
5  define -  := FMap.-            # alias for map removal
6  define at := FMap.at           # alias for map lookup
7
8  define-sort Clock := (FMap.Map Ide N)
9  define-sort Registry := (FMap.Map Ide Clock)
10
```

```
define clock-empty := (FMap.empty-map : Clock)

define (clock-get clock id) :=
    match clock {
        FMap.empty-map => zero
    | (FMap.update (pair key value) rest) =>
        (if (equal? key id)
            value
            (clock-get rest id))
    }
```

Snippet 3.1: Definitions inside module `VCCore`

Reading and updating. Missing components are C mapped to 0, matching the sparse vector convention in the literature [3, 9]. The local *tick* in 3.1 implements the advance on event rule.

```
define (clock-set clock actor value) :=
    match clock {
        FMap.empty-map => (FMap.update (pair actor value) FMap.empty-map)
    | (FMap.update (pair actor _) rest) => (FMap.update (pair actor value
        ) rest)
    | (FMap.update (pair other existing) rest) =>
            (FMap.update (pair other existing) (clock-set rest actor
                value))
    | _ => (FMap.update (pair actor value) FMap.empty-map)
    }

define (clock-tick clock actor) :=
    (clock-set clock actor (S (clock-get clock actor)))
```

Snippet 3.2: Local updates and tick

Join and order. 3.3 mirrors the pointwise max join and $\leq$ order (3.1), i.e., the algebra that CRDT merges rely on for inflation, idempotence, and commutativity [15].

```
define (clock-join source target) :=
    match source {
        FMap.empty-map => target
    | ([actor count] ++ rest) =>
            (let ((merged (clock-join rest target)))
                (let ((existing (clock-get merged actor)))
                    (clock-set merged actor (clock-max count existing))))
    | _ => target
    }

define (clock-leq left right) :=
    match left {
        FMap.empty-map => true
    | ([actor count] ++ rest) =>
            (if (N.<= count (clock-get right actor))
                    (clock-leq rest right)
                    false)
    | _ => true
    }
```

Snippet 3.3: Join and order

Registries. Per-actor snapshots capture what each actor knows about each other.

```
define registry-empty := (FMap.empty-map : Registry)

define (registry-get registry actor) :=
    match registry {
        FMap.empty-map => clock-empty
    | (FMap.update (pair actor clock) rest) => clock
    | (FMap.update (pair other clock) rest) => (registry-get rest actor)
    | _ => clock-empty
    }

```

```
define (registry-set registry actor clock) :=
    match registry {
        FMap.empty-map => (FMap.update (pair actor clock) FMap.empty-map)
    | (FMap.update (pair actor _) rest) => (FMap.update (pair actor clock
        ) rest)
    | (FMap.update (pair other existing) rest) =>
            (FMap.update (pair other existing) (registry-set rest actor
                clock))
    | _ => (FMap.update (pair actor clock) FMap.empty-map)
    }

} # end module VCCore
```

Snippet 3.4: Registries of clocks (anti-entropy metadata)

3.3 From algebra to code

More at the top of the previous section

Table 3.1: Mathematics ↔ implementation

Mathematical notion	Code realisation
Clock $C : \mathsf{Ide} \to \mathbb{N}$ (finite support)	`define-sort Clock := (FMap.Map Ide N)`
Absent component $\equiv 0$	`clock-get` returns `zero` on miss
Join ($\sqcup$) as componentwise max	`clock-join` with `clock-max`
Order ($\preceq$) as pointwise $\leq$	`clock-leq`
Local event at p	`clock-tick clock p`
Knowledge cache / anti-entropy	`Registry` (`registry-get`/`registry-set`)

3.4 Verified fragments

We prove structural facts by rewriting over finite maps. These correspond to the semilattice obligations used later in CRDT convergence proofs [15].

```
conclude clock-get-empty-zero :=
(forall ?actor:Ide .
    (clock-get clock-empty ?actor) = zero)

conclude clock-set-empty-overwrites :=
(forall ?actor:Ide ?value:N .
    (= (at (clock-set clock-empty ?actor ?value) ?actor)
         (SOME ?value)))

conclude clock-set-shadow-updates :=
(forall ?actor:Ide ?old:N ?new:N .
    (= (at (clock-set (clock-set clock-empty ?actor ?old)
                                             ?actor
                                             ?new)
                      ?actor)
         (SOME ?new)))
```

Snippet 3.5: Sample lemmas about `VCCore` operations

From these, we may derive (omitted here): commutativity/associativity/idempotence of `clock-join`; partial-order laws for `clock-leq`; inflation $C \preceq C \sqcup D$ and $D \preceq C \sqcup D$; and the strict HB test $C \prec D \Leftrightarrow \big(C \preceq D \wedge \exists i.\, C(i) < D(i)\big)$. These are exactly the algebraic approaches used by state-based CRDT proofs [15].

Have to do: what this? bad title

3.5 Why this matters for state-based CRDTs

Have to do: build more on the properties, show an example of the problem and then how the formalization solves the problem. This is your validation, Actually put this

as a separate section Attaching clocks to join-semilattice payloads enables:

- **Dominance checks:** if $C_{\text{in}} \preceq C_{\text{local}}$, skip the payload join (no new knowledge).
- **Inflationary metadata:** metadata joins with $\sqcup$; payload joins with its lattice $\sqcup_{\text{pay}}$. Both of them are idempotent, commutative, associative.
- **Delta subsumption:** registries summarise what each actor knows about each other.

4 CRDT Semantics and the Payload Framework

4.1 Introduction

Conflict-free Replicated Data Types (CRDTs) are data structures that allow multiple replicas to be updated concurrently while guaranteeing eventual convergence. The payload state of a CRDT forms a join-semilattice, and the update operations are monotonic with respect to its partial order. Shapiro *et al.* formalized this model and showed that if the state merge operation is commutative, associative and idempotent (a join) and the update function monotonically increases the state, then replicas that process the same set of updates (probably in different orders) will reach an identical state [15]. In this chapter we introduce the `PayloadFramework`, our Athena module that captures these semantics in a reusable form.

4.2 Mathematical model

Let $(S, \sqcup)$ be a join-semilattice: a set S equipped with a binary operation $\sqcup : S \times S \to S$ that is commutative, associative and idempotent. It induces a partial order $\leq$ on S by the rule

$$x \leq y \iff x \sqcup y = y.$$

This makes $(S, \leq)$ a partially ordered set with least upper bounds: for any $x, y \in S$, $x \sqcup y$ is the least element z such that $x \leq z$ and $y \leq z$. A CRDT's *state* lives in such a semilattice. The *merge* of two replica states corresponds to $\sqcup$; it must therefore be commutative ($x \sqcup y = y \sqcup x$), associative ($x \sqcup (y \sqcup z) = (x \sqcup y) \sqcup z$) and idempotent ($x \sqcup x = x$). Updates are modelled by a

function $\mathsf{apply} : S \times U \to S$, where U is the set of update operations. To guarantee convergence, updates must be monotonic: for all $s \in S$ and $u \in U$, we require $s \leq \mathsf{apply}(s, u)$ [15]. This ensures that applying the same update multiple times does not revert the state or break convergence.

4.3 The `PayloadFramework` module

The `PayloadFramework` module declares two domains, `State` and `Update`, representing the type of the CRDT's state and update respectively. It then introduces the merge function `merge`, the order relation `leq`, and the update function `apply`, together with axioms capturing the semilattice and monotonicity requirements.

```
1  module PayloadFramework {
2  declare merge: [State, State] -> State
3  declare leq: [State, State] -> Bool
4
5  assert [merge-comm] := (forall x y . merge(x,y) = merge(y,x))
6  assert [merge-assoc] := (forall x y z . merge(x, merge(y,z)) = merge(
       merge(x,y), z))
7  assert [merge-idem] := (forall x . merge(x,x) = x)
8  assert [merge-order] := (forall x y . leq(x,y) <-> merge(x,y) = y)
9
10 declare apply: [State, Update] -> State
11 assert [update-monotone] := (forall s u . leq(s, apply(s,u)))
12 }
```

Snippet 4.1: Abstract specification in `PayloadFramework`

Semantics. These definitions state that `merge` is a commutative, associative, idempotent operation with `leq` as the induced order: `merge(x,y) = y` exactly when `leq(x,y)`. The axiom `update-monotone` requires that applying an update never decreases the state. Together, these conditions ensure that the state space (`State`, `leq`, `merge`) forms a join-semilattice and that `apply` is monotonic. Shapiro *et al.* proved that any state-based CRDT satisfying these axioms is *strongly eventually consistent*: replicas that see the same set of updates converge to the same

state [15].

4.3.1 Instantiating the framework

The `Payload` module provides a thin wrapper for instantiating the abstract `PayloadFramework`. A payload is represented by a structure `PayloadOps` that carries its state and update types, along with defining equations for the update and merge functions. More precisely, a payload instance `(payload-ops init)` stores the initial state `init`, and two abstract operations `payload-step` and `payload-join` must be defined for that payload. The module's interface can be summarised as follows:

```
module Payload {
  structure (PayloadOps State Value) := (payload-ops State)

  declare payload-step: (State, Value)
    [(PayloadOps State Value) Value State] -> State
  declare payload-join: (State, Value)
    [(PayloadOps State Value) State State] -> State

  define (payload-apply ops value state) :=
    (payload-step ops value state)
}
```

Snippet 4.2: Abbreviated interface of the `Payload` module

4.3.2 Payload initialisation

For ease of use the framework provides helper functions that normalise a payload descriptor and extract its initial state. These definitions do not change the semantics of a payload. Instead, they provide a uniform way to pattern-match on the descriptor and recover the initial state inside generic operations. The following snippet illustrates how a payload descriptor is normalised and initialised:

```
define (payload-normalize ops) := ops
```

```
2
3  define (payload-canonical-from-init ops init) := ops
4
5  define (payload-canonical ops) :=
6    match (payload-normalize ops) {
7      (payload-ops init) => (payload-canonical-from-init ops init)
8    }
9
10 define (payload-init ops) :=
11   match (payload-normalize ops) {
12     (payload-ops init) => init
13   }
```

Snippet 4.3: Payload initialisation

Here `payload-normalize` is currently the identity function, but it exists so that more complex payload descriptors could normalise structural variation in the future. The function `payload-canonical` uses pattern matching to expose the initial state from a `PayloadOps` constant by delegating to `payload-canonical-from-init`. Finally, `payload-init` simply returns this initial state. Together, these helpers ensure that every payload descriptor carries the necessary information to initialise actors uniformly.

In this interface, a CRDT is represented by a `PayloadOps` constant encapsulating its initial state. A concrete CRDT defines equations for `payload-step` and `payload-join` by matching that constant. For example the grow-only set defines `payload-step` as set insertion and `payload-join` as set union. A grow-only counter defines `payload-step` as increment and `payload-join` as the maximum. The wrapper `payload-apply` is a convenience for applying updates during message reduction.

4.3.3 Actor configurations and message processing

The `Payload` module defines datatypes and functions to build actor configurations with payload states and message networks. A configuration carries a map from actor identifiers to their local

states and a multiset of pending messages. By encapsulating these structures and utilities in a reusable module, we can reason about message passing while focusing on the algebraic properties of merge and update.

Map and message abstractions. We first define convenience aliases for empty maps and multisets and overload some operations for finite maps and multisets. We then introduce the `Message` datatype (a tagged triple consisting of a destination identifier, a payload and some metadata) and the `Configuration` datatype, which bundles an actor state map and a message network. Two simple selectors extract the actors and the network from a configuration. All of these definitions fit on a few lines:

```
datatype (Message Value Meta) := (msg Ide Value Meta)

declare empty-message-net: (State, Value)
  [(PayloadOps State Value)] -> (MSet.MSet (Payload.Message Value Payload
      .Unit))
define (empty-message-net ops) := MSet.null

datatype (Configuration State Value Meta) :=
  (kfg am:(FMap.Map Ide State) net:(MSet.MSet (Message Value Meta)))

define (actors cfg) :=
  match cfg {
    (Payload.kfg am _) => am
  }

define (network cfg) :=
  match cfg {
    (Payload.kfg _ net) => net
  }
```

Snippet 4.4: Message and configuration utilities

The type `Message` encapsulates a destination identifier, a payload and optional metadata. A `Configuration` pairs an actor state map with a multiset of pending messages. The selector functions `actors` and `network` simply return the state map and the message multiset, respectively.

Lookup and message enabling. We next define two helpers used internally by the message processing logic. The first, `map-lookup`, searches for a key in a finite map and returns `SOME` of the associated value if found, or `NONE` otherwise. This function is analogous to the associative array lookup in standard data structures. It is defined by structural recursion on the map:

```
define (map-lookup map target) :=
  match map {
    FMap.empty-map => NONE
  | (FMap.update (pair key value) rest) =>
      match (equal? key target) {
        true => (SOME value)
      | false => (map-lookup rest target)
      }
  }
```

Snippet 4.5: Map lookup helper

The second helper, `Enabled`, predicates whether there is at least one deliverable message in a configuration. It returns `false` if the message multiset is empty, and otherwise checks that the message's destination actor exists in the actor map. This avoids delivering messages to nonexistent actors:

```
define (Enabled cfg) :=
  match cfg {
    (Payload.kfg am empty-mset) => false
  | (Payload.kfg am (MSet.insert (msg id _ _) rest)) =>
      match (map-lookup am id) {
        NONE => false
      | (SOME _) => true
      }
```

```
  | _ => false
  }
```

Snippet 4.6: Enabled predicate

The predicate `Enabled` therefore returns `true` exactly when there is at least one pending message whose destination actor is known, signalling that the system can make progress by delivering a message.

Message reduction. The core of the execution model is the `reduce` function, which delivers a single message and updates the actor state accordingly. If the message network is empty, the configuration is returned unchanged. Otherwise we remove one message `(msg id payload meta)` from the multiset, look up the current state of the target actor, and, if the actor exists, apply the payload and update the state map. Formally:

```
define (reduce ops cfg) :=
  match cfg {
    (Payload.kfg am empty-mset) => cfg
  | (Payload.kfg am (MSet.insert (msg id payload _) rest)) =>
      match (map-lookup am id) {
        NONE => (Payload.kfg am rest)
      | (SOME current) =>
          (Payload.kfg (FMap.update (pair id (payload-apply ops payload
              current)) (am - id))
                                rest)
      }
  | _ => cfg
  }
```

Snippet 4.7: Reduction of a single message

Enqueueing new messages. To schedule updates for delivery, the helpers `enqueue-with` and `enqueue` add messages to the network. The first allows attaching an arbitrary metadata tag, while the second simply uses the unit tag. These are straightforward constructors:

```
define (enqueue-with cfg id payload meta) :=
  (Payload.kfg (am cfg)
               (MSet.insert (msg id payload meta)
                            (net cfg)))

define (enqueue cfg id payload) :=
  (enqueue-with cfg id payload unit)
```

Snippet 4.8: Enqueueing updates for delivery

Initialising actor maps. Finally, the helper `init-map` builds an initial state map given a list of actor identifiers by assigning each actor the payload's initial state. The `init` function combines this map with an empty message set to produce a starting configuration. We explicitly declare the type of `init` to emphasise that it returns a configuration:

```
define (init-map ops ids) :=
  match ids {
    nil => empty-map
  | (id :: rest) =>
    (FMap.update (pair id (payload-init ops)) (init-map ops rest))
  | _ => empty-map
  }

declare init: (State, Value)
  [(PayloadOps State Value) (List Ide)] -> (Configuration State Value
      Payload.Unit)

define (init ops ids) :=
  (Payload.kfg (init-map ops ids)
                        (empty-message-net ops))
```

Snippet 4.9: Initialising actor maps and configurations

The helper `init-map` traverses the list of actors and uses `payload-init` to seed each actor with an initial state. The resulting map is paired with an empty message network by `init` to form a configuration ready for execution.

4.4 Properties and verification

The axioms embedded in `PayloadFramework` guarantee strong eventual consistency for any CRDT that instantiates the module. In particular, one proves that the merge function is a join: it yields the least upper bound of its arguments with respect to `leq`. By monotonicity, updates move the state upwards in the lattice. These two facts are sufficient to show that if two replicas process the same multiset of updates (possibly in different orders), their final states are equal.

5 Replication Backbone and Causal Consistency

5.1 Introduction

The *CRDTBackbone* module realises a generic operation based replication protocol. It alligns the vector clock definitions from Chapter **??** with the payload semantics from Chapter 4. By associating each update with a timestamp and enforcing causal delivery. This chapter explains the organisation of the backbone, illustrates its main algorithms and sketches why they are correct.

5.2 Configurations and vector clocks

The backbone maintains, for each replica, its local payload state, a vector clock summarising the set of updates it has seen, and a network buffer of pending messages. Concretely, a configuration comprises three components:

- A finite map *states* : $\mathsf{Ide} \to S$ mapping each replica identifier to its current payload state (with S the payload's state domain);
- A *registry clocks* : $\mathsf{Ide} \to VC$ storing, for each replica, its current vector timestamp [7, 3, 9];
- A multiset *net* of messages, each message being a triple $(\text{destination}, \text{payload}, \text{timestamp})$.

5.2.1 Configuration type and basic accessors

In Athena we model a configuration using a record with three fields: a map of actor states, a registry of vector clocks, and a multiset of messages. The following snippet shows the type definition and simple accessor functions:

```
datatype (Configuration State Value) :=
  (kfg astates:(FMap.Map Ide State)
       clocks:Registry
       net:(MSet.MSet (Message Value Clock)))
```

Snippet 5.1: Configuration datatype

```
define (get-cfg-states cfg) :=
  match cfg {
    (Backbone.kfg states _ _) => states
  }

define (get-cfg-clocks cfg) :=
  match cfg {
    (Backbone.kfg _ clks _) => clks
  }

define (get-cfg-net cfg) :=
  match cfg {
    (Backbone.kfg _ _ network) => network
  }
```

Snippet 5.2: Configuration accessors

The record constructor `Backbone.kfg` encapsulates three components together. The helper functions `get-cfg-states`, `get-cfg-clocks` and `get-cfg-net` simply select the appropriate field. These accessors are used throughout the backbone to inspect the configuration without showing its complete structure.

5.3 The CRDTBackbone module

We introduce its core functions in four groups: local clock management, message delivery, broadcasting, and initialisation. For clarity we omit low-level helper functions such as finite map constructors and list utilities, focusing instead on the semantics.

5.3.1 Local ticks and vector clocks

When a replica performs a local update, it must advance its vector clock. The function `cfg-tick` returns a new configuration with the caller's clock incremented accordingly, leaving the state and network unchanged. The helper `cfg-actor-clock` retrieves the current clock of a given actor. Both functions are defined by pattern matching on the configuration record:

```
1  define (cfg-tick cfg actor) :=
2    (Backbone.kfg (get-cfg-states cfg)
3                  (registry-tick (get-cfg-clocks cfg) actor)
4                  (get-cfg-net cfg))
5
6  define (cfg-actor-clock cfg actor) :=
7    (registry-get (get-cfg-clocks cfg) actor)
```

Snippet 5.3: Updating and querying vector clocks

Invoking `cfg-tick cfg actor` produces a new configuration in which only the vector clock registry is updated: the entry for `actor` is incremented using `registry-tick`. This corresponds to the local event of generating a new update. The function `cfg-actor-clock` simply looks up the replica's clock in the registry; it is used when constructing messages to find the correct timestamp.

5.3.2 Message delivery

The `reduce` function models the delivery of a single message. If the message queue is empty, the configuration is returned unchanged. Otherwise, one message `(msg id payload clock)` is removed from the multiset. The recipient's current state is obtained via `cfg-state-lookup`; the

payload update is applied using `payload-apply`; and the recipient's clock registry is updated by merging in the incoming clock via `registry-merge`. A new configuration with the updated state map, clock registry and reduced message network is returned. The helper `reduce*` simply applies `reduce` until no messages remain. These functions implement exactly-once message delivery and are vital for ensuring idempotence:

```
define (reduce ops cfg) :=
  match (get-cfg-net cfg) {
    MSet.null => cfg
  | (MSet.insert m rest) =>
      match m {
        (Payload.msg id payload clock) =>
          let ((states (get-cfg-states cfg))
               (clocks (get-cfg-clocks cfg))
               (current (cfg-state-lookup states id (payload-init ops))
                   )
               (updated (payload-apply ops payload current))
               (next-states (FMap.update (pair id updated) states))
               (next-clocks (registry-merge clocks id clock)))
          in
            (Backbone.kfg next-states next-clocks rest)
      }
  }

define (reduce* ops cfg) :=
  match (get-cfg-net cfg) {
    MSet.null => cfg
  | (MSet.insert _ _) => (reduce* ops (reduce ops cfg))
  }
```

Snippet 5.4: Message delivery and repetition

The check `cfg-state-lookup states id (payload-init ops)` returns a default initial state if the recipient has not yet been initialised; this makes the backbone robust to late joins. After

applying the payload, the vector clock registry is updated via `registry-merge`, which computes the componentwise maximum. Subsequent messages carrying smaller or equal timestamps will be discarded by the causal ordering checks in `reduce`. The `reduce*` function is used to reduce a configuration to a quiescent state where no messages remain.

5.3.3 Broadcasting updates

When a replica performs a local update, the backbone must propagate this operation to all other replicas. The helper `enqueue-message` inserts a message into the network buffer, and `enqueue-over-states` iterates over all actors, enqueuing a message for each. The function `broadcast-from` combines these primitives: it first ticks the sender's clock, applies the update locally, computes the new vector clock, and then enqueues messages for all actors with that timestamp. Finally, `broadcast` sends an update with the clock set to the zero vector (for use by pure state-based CRDTs or out-of-band operations). A condensed version of these functions is given below:

```
1  define (enqueue-message cfg recipient payload clock:Clock) :=
2    (Backbone.kfg (get-cfg-states cfg)
3                  (get-cfg-clocks cfg)
4                  (MSet.insert (Payload.msg recipient payload clock)
5                               (get-cfg-net cfg)))
6
7  define (enqueue-over-states cfg states payload clock-fn) :=
8    match states {
9      FMap.empty-map => cfg
10   | (FMap.update (pair actor _) rest) =>
11       let ((next (enqueue-message cfg actor payload (clock-fn actor))))
12         in (enqueue-over-states next rest payload clock-fn)
13   }
```

Snippet 5.5: Enqueueing updates

The helper `enqueue-message` inserts a single message into the network buffer without applying it. The function `enqueue-over-states` iterates over all actors in the state map, invoking `enqueue-message` for each. These functions schedule updates for later delivery.

```
1  define (broadcast-from cfg actor payload) :=
2    let ((advanced (cfg-tick cfg actor))
3         (clock (cfg-actor-clock advanced actor)))
4      in (enqueue-over-states advanced (get-cfg-states cfg)
5                              payload (lambda (_) clock))
6
7  define (broadcast cfg payload) :=
8    (enqueue-over-states cfg (get-cfg-states cfg)
9                         payload (lambda (_) clock-empty))
```

Snippet 5.6: Broadcast functions

In `broadcast-from`, the sender's clock is advanced via `cfg-tick`, the update is applied locally, and the resulting timestamp is attached to messages enqueued for every actor. The `broadcast` function uses a zero vector clock for situations where timestamps are not required.

5.3.4 Initialisation and progress conditions

We must create a configuration with initial states and clocks for each actor. The helper `init-state-map` builds a map from a list of actor identifiers to the payload's initial state, while `init-clock-map` constructs a registry of zero vector clocks. The `init` function then packages these into a configuration with an empty message queue. Here is a summary of these functions:

```
1  define (init-state-map ops ids) :=
2    match ids {
3      nil => empty-map
4    | (id :: rest) =>
5        (FMap.update (pair id (payload-init ops)) (init-state-map ops
             rest))
6    }
7
8  define (init-clock-map ids) :=
9    match ids {
```

```
      nil => registry-empty
    | (id :: rest) =>
        (FMap.update (pair id clock-empty) (init-clock-map rest))
    }
```

Snippet 5.7: Initialising state and clock maps

```
define (init ops ids) :=
  (Backbone.kfg (init-state-map ops ids)
                (init-clock-map ids)
                empty-mset)

define (reducible cfg) :=
  match cfg {
    (Backbone.kfg _ _ pending) => (net-has-pending pending)
  }

define (deadlocked cfg) :=
  match cfg {
    (Backbone.kfg _ _ pending) => (net-empty pending)
  }
```

Snippet 5.8: Configuration initialisation and progress predicates

The predicate `reducible cfg` returns `true` whenever there is at least one message awaiting delivery. Conversely, `deadlocked cfg` holds when the message network is empty. Variants such as `progress` and `livelock` can be defined in terms of these primitives to reason about termination and fairness; we omit them here for brevity.

5.4 Properties and verification

Message passing correctness depends on a combination of vector-clock ordering and semilattice semantics. We show the key properties informally.

1. **Causal delivery.** A message tagged with a vector clock vc is only applied once all messages with strictly smaller timestamps have been delivered at the receiving replica. This follows from the check `vc <= myClock` in `reduce`: if the incoming clock is not greater than the local clock in some component, the message is treated as redundant and discarded. The correctness of this check relies on the partial order properties of vector clocks[7, 3, 9].

2. **No duplication.** Each update is applied at most once at each replica. When a message (id, u, vc) is processed, the recipient's clock is updated to the join of its current clock and vc. Any later message with timestamp vc' such that $vc' \leq vc$ will satisfy the causal check and be ignored. Duplicate suppression is therefore implemented via the vector-clock join[7, 3, 9].

3. **Convergence.** If two replicas eventually process the same set of update operations—possibly in different orders—their payload states become identical. This property combines the join-semilattice semantics of the payload (commutativity, associativity and idempotence of `merge`, and monotonicity of `apply`) with the causal delivery and duplicate suppression enforced by the backbone. Since the operations commute and the reduction order does not matter, the final states after `reduce*` are equal. This is the essence of Strong Eventual Consistency (SEC)[15], and it illustrates the equivalence between state-based and operation-based viewpoints[15].

One theorem states that for any payload operations `ops` and any configuration `cfg`, if two replicas have delivered the same multiset of messages, then `reduce* ops cfg` yields equal states for those replicas. The proof proceeds by induction on the size of the message multiset and uses the commutativity and idempotence of both `payload-apply` and `registry-merge`. Liang and Feng's ACC methodology stresses this separation of concerns: one first verifies convergence (SEC) for the CRDT's payload and backbone, then proves functional correctness with respect to a sequential specification[8]. Our design follows this pattern.

5.5 Conclusion

The `CRDTBackbone` module isolates the concerns of network replication from the algebraic behaviour of a CRDT. By combining vector-clock metadata with a simple message scheduler, it implements the assumptions required by operation-based CRDTs: reliable causal delivery and exactly-once application of commutative updates[15]. When composed with a payload satisfying the semilattice axioms and monotonicity from Chapter 4, the backbone yields a replicated data type that converges regardless of network reorderings or duplicates.

6 Validation and Verification

6.1 Introduction

This chapter validates the Athena CRDT framework along two topics: (1) formal proofs over the *payload*, *vector clocks*, and *backbone* layers that establish safety properties; and (2) executable demos that witness end-to-end behaviour for representative CRDTs. Our goal is to show that these abstractions introduced in chapters 4, 3, and 5 support concise proofs.

We focus on three key properties: - **Convergence (SEC)** for state-based CRDTs: replicas that process the same set of updates (order-insensitively) reach identical states [15]. - **Progress**: progress holds when deliverable messages exist. - **Livelock characterisation**: livelock implies progress (i.e., there are pending messages but the system is stuck repeating a configuration), which is useful to separate fairness assumptions from safety.

Notation. We write $\mathcal{C}$ for configurations, $\mathsf{net}(\mathcal{C})$ for the multiset of pending messages, and $\mathsf{reduce}^*(\mathcal{C})$ for the maximal delivery closure. Vector clocks use the partial order $\preceq$ and join $\sqcup$ (componentwise max) as in Chapter 3 [7, 3, 9].

6.2 Backbone liveness

The backbone layer (Chapter 5) provides generic configuration plumbing. We show the main definitions and liveness facts in small snippets.

6.2.1 Configuration and reduction

We first recall how a backbone configuration is represented as a record of states, clocks, and a multiset of pending messages. This is precisely the structure that all of our CRDTs manipulate.

```
load "../src/framework/backbone.ath"
open Backbone

datatype (Configuration 'State 'Value) :=
  (kfg states:(FMap.Map Ide 'State)
       clocks:Registry
       net:(MSet.MSet (Payload.Message 'Value Clock)))

define (reduce ops cfg) :=
  match (next-message cfg) {
    (none)      => cfg
  | (some msg) => (deliver ops msg cfg)
  }
```

Snippet 6.1: Backbone configuration and reduction

The datatype `Configuration` abstracts the concrete CRDT state and value types and exposes the network as a multiset of messages. The function `reduce` is a single delivery step: it leaves the configuration as it was when there is no pending messages or it delegates to `deliver`.

6.2.2 Net draining and deadlock

The shared proof module textttbackbone_props.ath specialises this generic interface and proves that repeated reduction drains the network and characterises deadlock.

```
define reduce*-drains-net :=
  (forall ?init:'State ?states:(FMap.Map Ide 'State) ?clocks:Registry
    (forall ?pending:(MSet.MSet (Payload.Message 'Value Clock))
      (= (get-cfg-net (reduce* (payload-ops ?init)
```

```
                                (Backbone.kfg ?states ?clocks ?pending)))
        MSet.null)))
```

Snippet 6.2: Backbone: net draining and deadlock

The above definition states that applying `reduce*` (the reflexive–transitive closure of `reduce`) to any finite multiset of messages eventually empties the network. It allow us to reason about CRDT behaviour: after delivering all messages, either we are deadlocked (no more work) or there exists a deliverable message.

6.2.3 Progress and livelock

Progress and livelock are also phrased using the generic `Configuration` type, independently of any particular CRDT.

```
define progress-equals-reducible :=
  (forall ?S ?R ?N . (= (progress (Backbone.kfg ?S ?R ?N))
                        (reducible (Backbone.kfg ?S ?R ?N))))

define livelock-implies-progress :=
  (forall ?ops ?S ?R ?N .
     ((livelock ?ops (Backbone.kfg ?S ?R ?N))
       ==> (progress (Backbone.kfg ?S ?R ?N))))
```

Snippet 6.3: Backbone: progress and livelock

The first equation connects the abstract notion of progress to the operational predicate `reducible`, defined in terms of `reduce`. The livelock lemma isolates the role of fairness: if the system is in a livelock, there is still progress in terms of reducible configuration, but termination may require assumptions.

6.3 CRDT case studies

We now instantiate the framework with concrete payloads drawn from the CRDTs that have working demos: GCounter, GSet, OR-Set, and the non-negative counter. Each case shows

one or two core definitions or properties and explains how they are expressed using the shared backbone.

6.3.1 Grow-only counter (GCounter)

The GCounter demo (‘gcounter$_d$emo.ath‘)$buildsacounterbyusingamapfromactoridentifierstonaturalnumbersa$

```
open GCounter

datatype State := (st value:(FMap.Map Ide N))

define (merge s1 s2) :=
  (st (FMap.map-union-with max (value s1) (value s2)))

define gcounter-ops :=
  (payload-ops State Value merge apply-update)
```

Snippet 6.4: GCounter payload and backbone instance

The `State` type and `merge` function form a join-semilattice where the join is computed pointwise with `max`. The helper `gcounter-ops` packages this structure into a `payload-ops` record, which the backbone then uses to define generic configuration and reduction functions. This pattern is seen across all our CRDTs, the framework expects a small algebraic interface rather than ad-hoc code.

On top of these definitions, the GCounter property module shows how to express “deliver all messages” using the `reduce*` operation.

```

define (deliver-all cfg) :=
  (reduce* gcounter-ops cfg)

define deliver-all-drains-net :=
  (forall ?cfg:(Backbone.Configuration GCounter.State GCounter.Value)
    (= (get-cfg-net (deliver-all ?cfg)) MSet.null))
```

Snippet 6.5: GCounter: deliver-all drains network

Here, `deliver-all` is just `reduce*` instantiated with the GCounter payload. `deliver-all-drains-net` is a direct consequence of the generic net-draining property, showing how CRDT-specific theorems can be phrased with almost no boilerplate.

6.3.2 Grow-only set (GSet)

The GSet demo (‘gset$_{d}emo.ath$‘)$isanotherminimalinstantiation : thestateisasetofelements, mergeissetunion, andupda$

```
open GSet

datatype State := (st els:(Set.Set Elem))

define (merge s1 s2) :=
  (st (Set.union (els s1) (els s2)))
```

Snippet 6.6: GSet payload

This code mirrors the GCounter setup, but swaps maps for sets. The framework does not need to change: the same backbone and progress lemmas apply, once we provide the appropriate `merge`.

6.3.3 Non-negative counter (NP-Counter)

The non-negative counter demo (‘np$_{c}ounter_{d}emo.ath$‘)$usesaslightlyricherpayloadtoenforcethatdecrementsnevermaket$

```
open NPCounter

datatype State :=
  (np-state pos:(FMap.Map Ide N)
            neg:(FMap.Map Ide N))

define (value st) :=
  (FMap.sum (pos st) - FMap.sum (neg st))
```

Snippet 6.7: NP-Counter payload

The `State` type splits the contribution of increments and decrements, while the function `value` computes the associated integer. The merge function is again a pointwise maximum on each component; this guarantees monotonicity of both `pos` and `neg`, and hence of the overall `State`. The backbone treats this CRDT exactly like the GCounter, illustrating that richer invariants can be enforced entirely at the payload level.

6.3.4 OR-Set and vector clocks

The OR-Set demos (‘orset$_demo.ath$‘and‘orset$_vc_demo.ath$‘)showcasethepartoftheframeworkthatusesvectorclock
onefor additionsand, intheVCvariant, onefor removals.

```
datatype Entry :=
  (entry elem:Elem
         add-clock:Clock
         rem-clock:Clock)

define (visible? e clock) :=
  (~ (clock-happens-before (rem-clock e) clock))
```

Snippet 6.8: OR-Set entry structure

The `Entry` datatype records both the addition and removal clocks. The predicate `visible?` states that an entry remains visible at a given replica clock as long as the removal clock does not happen before the replica's clock—that is, the removal has not been causally applied everywhere. This definition relies directly on the vector clock primitives of Chapter 3.

The VC-based OR-Set also reuses the same backbone configuration and reduction operations. Only the payload (entries, clocks, and visibility predicate) changes.

6.4 Equational characterisations

We summarise the key equations that the implementations satisfy: - **Vector clocks:** $C \preceq D \iff \forall i.\ C(i) \leq D(i)$ and $(C \sqcup D)(i) = \max\{C(i), D(i)\}$. - **Payload monotonicity:** $s \leq \mathsf{apply}(s, u)$ ensures inflation of state and idempotence under replay [15]. - **Net draining:** $\mathsf{net}(\mathsf{reduce}^*(\mathcal{C})) = \varnothing$. - **Progress/Deadlock:** $\mathsf{progress}(\mathcal{C}) \equiv \neg\, \mathsf{deadlocked}(\mathcal{C}) \equiv \mathsf{net}(\mathcal{C}) \neq \varnothing$.

These align directly with the code equations in `VCCore`, `Payload`, and `Backbone`.

6.5 Executable validation

We validate behaviour via Athena demos drawn from the CRDTs above. Each demo exercises the key behaviours of its CRDT.

Try it. From the repository root:

```
athena-run src/demos/crdts/gcounter_demo.ath
athena-run src/demos/crdts/gset_demo.ath
athena-run src/demos/crdts/np_counter_demo.ath
athena-run src/demos/crdts/orset_demo.ath
athena-run src/demos/crdts/orset_vc_demo.ath
```

Snippet 6.9: OR-Set entry structure

For each, the network is reduced to inactivity (`reduce*`); this confirms convergence and in NP-Counter and OR-Set, the expected interaction between payload invariants and vector-clock metadata.

6.6 Threats to validity

- **Fairness assumptions.** Livelock exclusion is not proved in general; the framework isolates it via `livelock-implies-progress` but fairness (or scheduling) must be supplied by the environment. - **Model fidelity.** The abstract network is a multiset; real transports may reorder, duplicate, or lose messages. Our proofs assume reliable delivery to match operation-based CRDT

requirements. - **Finite executions.** Net-draining arguments use structural recursion over finite multisets; infinite streams require reasoning not covered here.

6.7 Reproducibility

All modules and proofs are in the public GitHub repository at `https://github.com/FLAGlab/Athena-CRDTs`. Bibliographic entries reside in `thesis/bib/`. Code snippets in this chapter correspond to files under `src/framework` and `src/crdts`.

6.8 Conclusion

The validation shows that our implementation realises the algebraic CRDT model: payloads satisfy semilattice and monotonicity laws, vector clocks provide causal metadata, and the backbone composes them to ensure convergence.

7 Future Work

This thesis presents the basis for a CRDT verification framework in Athena. Future work will concentrate on expanding the framework to support a wider variety of CRDTs, enabling the formal verification of diverse consistency properties and operational behaviors across different distributed systems scenarios.

To achieve this, modular and extensible components will be developed within Athena to formalize the syntax, semantics, and proofs for operations common to CRDTs. These modules will reduce the effort required to implement new CRDTs while ensuring their correctness under various failure and adversarial conditions.

8 Conclusions

The development of the Athena framework focuses on the development of a CRDT verification framework within the Athena tool, addressing the growing need for formally verified consistency mechanisms in distributed systems. The framework provides a systematic approach to specifying, implementing, and verifying CRDTs, ensuring correctness challenging conditions.

The work contributes to advancing the formal verification in distributed systems by bridging the gap between theoretical consistency guarantees and practical verification tools.

Bibliography

The references are sorted alphabetically by first author.

[1] Péter Bereczky, Dániel Horpácsi, and Simon Thompson. A formalisation of core erlang, a concurrent actor language. *Acta Cybernetica*, 26(3):373–404, 2024. DOI 10.14232/actacyb.298977. URL `https://cyber.bibl.u-szeged.hu/index.php/actcybern/article/view/4333`.

[2] Lyn Dupré. *BUGS in Writing: A Guide to Debugging Your Prose*. Pub-addison, revised edition, 1998. ISBN 0-201-37921-X.

[3] Colin J. Fidge. Timestamps in message-passing systems that preserve the partial ordering. In *Proceedings of the 11th Australian Computer Science Conference (ACSC '88)*, volume 10 of *Australian Computer Science Communications*, pages 56–66, University of Queensland, Brisbane, Australia, February 1988. URL `https://classpages.cselabs.umn.edu/Spring-2018/csci8980/Papers/Foundations/fidge.pdf`. Vector clocks introduced independently (asynchronous and synchronous cases).

[4] Florian Jacob, Saskia Bayreuther, and Hannes Hartenstein. Byzantine behavior in crdts: Hashdag and matrix event graph. *SICHERHEIT 2022*, 2022. DOI 10.18420/SICHERHEIT2022_07. URL `http://dl.gi.de/handle/20.500.12116/40149`.

[5] Martin Kleppmann. Making crdts byzantine fault tolerant. In *9th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC'22)*, pages 1–8, 2022. DOI 10.1145/3517209.3524042. URL `https://doi.org/10.1145/3517209.3524042`.

[6] Shadaj Laddad, Conor Power, Mae Milano, Alvin Cheung, and Joseph M. Hellerstein. Katara: Synthesizing crdts with verified lifting. *Proc. ACM Program. Lang.*, 6:173, 2022. DOI 10.1145/3563336. URL `https://doi.org/10.1145/3563336`.

[7] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, July 1978. DOI 10.1145/359545.359563. URL `https://lamport.azurewebsites.net/pubs/time-clocks.pdf`.

[8] Hongjin Liang and Xinyu Feng. Abstraction for conflict-free replicated data types. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 636–650, 2021. DOI 10.1145/3453483.3454067. URL `https://dl.acm.org/doi/10.1145/3453483.3454067`.

[9] Friedemann Mattern. Virtual time and global states of distributed systems. In Michel Cosnard et al., editors, *Parallel and Distributed Algorithms: Proceedings of the International Workshop on Parallel and Distributed Algorithms (Chateau de Bonas, France, Oct. 1988)*, pages 215–226, Amsterdam, The Netherlands, 1989. North-Holland (Elsevier). URL `https://homes.cs.washington.edu/~arvind/cs425/doc/mattern89virtual.pdf`. Independent introduction of vector clocks; classic exposition of global states.

[10] David R. Musser and Carlos A. Varela. Structured reasoning about actor systems. In *Proceedings of the 2013 Workshop on Programming based on Actors, Agents, and Decentralized Control*, pages 37–48, 2013. DOI 10.1145/2541329.2541334. URL `https://dl.acm.org/doi/10.1145/2541329.2541334`.

[11] Kartik Nagar and Suresh Jagannathan. Automated parameterized verification of crdts. *arXiv preprint*, 1905.05684v1, 2019. URL `https://arxiv.org/abs/1905.05684v1`.

[12] Abel Nieto, Léon Gondelman, Alban Reynaud, Amin Timany, and Lars Birkedal. Modular verification of op-based CRDTs in separation logic. *Proceedings of the ACM on Programming Languages*, 6(OOPSLA2):1788–1816, October 2022. DOI 10.1145/3563351.

[13] Abel Nieto, Arnaud Daby-Seesaram, Léon Gondelman, Amin Timany, and Lars Birkedal. Modular verification of state-based crdts in separation logic. *arXiv preprint*, 2023.

[14] Kevin De Porre, Carla Ferreira, and Elisa Gonzalez Boix. Verifx: Correct replicated data types for the masses. *arXiv preprint*, 2207.02502, 2024. DOI 10.48550/arXiv.2207.02502. URL `http://arxiv.org/abs/2207.02502`.

[15] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Proceedings of the 13th International Conference on Stabilization, Safety, and Security of Distributed Systems*, pages 386–400, 2011.

[16] William Strunk and E.B. White. *The Elements of Style*. Longman, fourth edition, 2000. ISBN 0-205-30902-X.

[17] Christian Tschudin. A connectionless grow-only set crdt. In *Proceedings of the 3rd International Workshop on Distributed Infrastructure for the Common Good*, pages 25–30, 2022. DOI 10.1145/3565383.3566110. URL `https://dl.acm.org/doi/10.1145/3565383.3566110`.

[18] Carlos Varela and Gul Agha. Programming dynamically reconfigurable open systems with salsa. *ACM SIGPLAN Notices*, 36(12):20–34, 2001. DOI 10.1145/583960.583964. URL `https://dl.acm.org/doi/10.1145/583960.583964`.

[19] George Zakhour, Pascal Weisenburger, and Guido Salvaneschi. Type-checking crdt convergence. *Proc. ACM Program. Lang.*, 7:162, 2023. DOI 10.1145/3591276. URL `https://doi.org/10.1145/3591276`.

[20] Peter Zeller, Annette Bieniusa, and Arnd Poetzsch-Heffter. Formal specification and verification of crdts. *Formal Techniques for Networked and Distributed Systems*, pages 33–48, 2014. DOI 10.1007/978-3-662-43613-4_3. URL `https://inria.hal.science/hal-01398007`.